

# Mail-Server-Factory

**Gérez votre serveur de messagerie comme un pro —  
décrivez-le en JSON, déployez-le où vous voulez.**

## Résumé

Mail Server Factory est un outil de provisionnement de serveur de messagerie prêt pour la production et entièrement automatisé. L'utilisateur rédige une simple configuration JSON ; la Factory l'interprète et effectue toutes les installations et initialisations sur le système d'exploitation cible, déployant une pile de messagerie basée sur Docker et faiblement couplée, sur 12 types de connexions.

## Description courte

Un outil Kotlin/Shell qui transforme une description JSON en un serveur de messagerie entièrement installé et conteneurisé sous Docker. Il prend en charge 12 types de connexions (SSH, Docker, Kubernetes, AWS SSM, Azure, GCP, Libvirt, et bien d'autres), un cadre de sécurité complet, 25 distributions Linux, et est livré avec 439 tests validés.

## Description longue

Mettre en place un serveur de messagerie réel et sécurisé est l'un des rites de passage classiques en administration système — et l'un des plus notoirement pénibles. Postfix, Dovecot, les certificats TLS, les enregistrements DNS, les règles de pare-feu et les particularités propres à chaque distribution doivent tous s'aligner à la perfection, et une seule directive erronée peut entraîner des e-mails rejetés en silence ou un relais ouvert. Mail Server Factory prend l'ensemble de cette expertise durement acquise et sujette aux erreurs et l'incarne dans un logiciel. Au lieu de configurer manuellement chaque composant sur un système d'exploitation inconnu, l'utilisateur final décrit le résultat souhaité dans un simple document JSON ; la Factory lit ce fichier et exécute les étapes exactes d'installation et d'initialisation requises sur le

système d'exploitation cible, déployant une pile de messagerie fonctionnant sur Docker, où chaque composant est faiblement couplé — un choix de conception qui permet une scalabilité horizontale et autorise la mise à jour ou le remplacement de n'importe quel module de manière isolée. De plus, l'outil est délibérément agnostique en matière d'environnement : ses 12 types de connexions permettent d'utiliser le même outil et la même configuration JSON pour cibler une machine locale, un hôte distant via SSH, un runtime Docker ou Kubernetes, des instances cloud via AWS SSM / Azure Serial Console / GCP OS Login, ou des machines virtuelles via Libvirt — la même description déclarative, déployée là où vous le souhaitez. Il supporte 25 distributions Linux, couvrant les familles occidentales (Ubuntu, Debian, CentOS, Fedora, AlmaLinux, Rocky, openSUSE), russes (ALT, Astra, ROSA) et chinoises (openEuler, openKylin, Deepin), avec une installation sans surveillance via preseed/kickstart/cloud-init/autoyast et une automatisation de machines virtuelles basée sur QEMU pour les tests. Les fonctionnalités d'entreprise sont étendues : chiffrement AES-256-GCM, politiques de mots de passe et de clés SSH strictes, configuration automatique du pare-feu pour les ports de messagerie (25/587/465/993/995), TLS/SSL avec validation des certificats et HSTS, journalisation d'audit et RBAC. Les fonctionnalités opérationnelles incluent l'optimisation de la JVM (G1GC), le cache Caffeine, le pooling des connexions, des métriques compatibles Prometheus, la journalisation structurée, le rechargement à chaud de la configuration et la gestion des secrets. Le projet affiche 439 tests réussis à 100 % et une porte de qualité SonarQube irréprochable. Il est le fleuron de l'organisation Server-Factory.

## **Pourquoi nous l'avons conçu**

Configurer un serveur de messagerie sécurisé et prêt pour la production est une tâche notoirement sujette aux erreurs et dépendante du système d'exploitation. Mail Server Factory encapsule cette expertise dans un modèle déclaratif JSON associé à un moteur d'exécution, permettant ainsi de reproduire une pile de messagerie Dockerisée, correcte et sécurisée, sur n'importe quelle cible prise en charge, sans travail manuel étape par étape.

## **Pourquoi c'est révolutionnaire**

Il transforme le provisionnement d'un serveur de messagerie, autrefois une épreuve spécialisée de plusieurs jours nécessitant une précision absolue, en un simple acte de rédaction de configuration. De plus, il rend cette opération portable sur 12 types de connexions et 25 distributions Linux, tout en intégrant dès le départ des paramètres de sécurité d'entreprise. Le résultat est reproductible et *vérifiable* : le même JSON produit

systématiquement la même pile sécurisée, et les 439 tests réussis ainsi qu'un passage sans faille au contrôle SonarQube attestent que le moteur sous-jacent est soumis à des exigences strictes plutôt qu'à une confiance aveugle.

## Ce qui est innovant

- Modèle déclaratif JSON → installation/initialisation interprétée sur le système d'exploitation cible.
- 12 types de connexions (locale, SSH, Docker, Kubernetes, AWS SSM, Azure, GCP, Libvirt, et autres) gérés par un seul outil.
- Prise en charge de 25 distributions avec installation sans surveillance (preseed/kickstart/cloud-init/autoyast) et automatisation via QEMU.
- Pile Dockerisée faiblement couplée pour un scaling et des mises à jour indépendants.

## Défis et solutions

- **Hétérogénéité des OS/distributions** : résolue grâce à des recettes spécifiques par distribution, des configurations d'installation sans surveillance et des tests inter-distributions basés sur QEMU.
- **Atteinte de multiples cibles de déploiement** : résolue par 12 types de connexions modulables sous un moteur d'installation commun.
- **Sécurité par défaut** : assurée par le chiffrement AES-256-GCM, des politiques strictes pour les clés et mots de passe, des règles de pare-feu automatisées, ainsi que TLS/HSTS.
- **Confiance dans la fiabilité** : garantie par une suite de 439 tests (tous réussis) et un contrôle SonarQube sans faille.

## Pile technologique (pourquoi et comment)

- **Kotlin** — le moteur Factory et la logique des étapes d'installation (179 Ko ; Kotlin 2.0.21).
- **Shell** — scripts de provisionnement, gestionnaires ISO/QEMU et automatisation des OS (composant dominant en taille).
- **Docker** — environnement d'exécution de la pile de messagerie déployée et faiblement couplée.
- **QEMU** — automatisation des machines virtuelles pour l'installation et les tests inter-distributions.
- **JSON** — format de configuration déclaratif destiné à l'utilisateur.
- **Gradle 8.14.3 / Java 17** — chaîne d'outils de compilation.

- **Caffeine** — mise en cache multi-régions ; **JVM optimisée G1GC** pour les performances.
- **Métriques compatibles Prometheus** — supervision ; prêt pour Grafana/ELK.
- **Sieve** — règles de filtrage des emails (empreinte réduite dans les statistiques du langage).

Remarque : GitHub indique que le dépôt est un fork au sein de l'organisation Server-Factory. Précède la gamme de produits AI ; présenté comme un projet phare mature en DevOps/provisionnement, et non comme un utilitaire AI.